

Operating Systems Course - Laboratory Project

2026-04-27

1 Instructions

Please read the following instructions before starting the project.

- **Structure.** This project is worth 10 points out of the total grade of the course. As anticipated in the syllabus:
 - 20% of the grade will be based on the code you write: correctness, adherence to the specifications, code quality, etc.
 - 10% of the grade will be based on the report you write: clarity, completeness, presentation, etc.
 - 70% of the grade will be based on the presentation you will give at the exam.
- **Submission.** You **must** submit an archive that:
 - is named
 - * `${surname1}_${surname2}_${surname3}.tar.gz` after the surnames of the group members. For example, if the surnames of the group members are Rossi, Bianchi and Verdi, the file must be named `Rossi_Bianchi_Verdi.tar.gz`. The order of the surnames does not matter.
 - contains a report, called `report.pdf`, which contains a description of your project, the design choices you made, and any other information you think is relevant. The report must be in PDF format and must be no longer than 5 pages.
 - contains a folder called `code`, which contains the source code of your project, organized as you wish. The code must be written in C and Bash, and it must compile and run correctly on the laboratory computers (or any Ubuntu 24.04 machine).
- **Compilation.** The project must include a way to compile the code and produce the necessary executables to run the project. You can choose to use either:
 - A **Makefile**, which compiles the code and produces the necessary executables to run the project. The Makefile must support at least the following targets:
 - * **build**: compiles the project and produces the files required to run the project (check below for the required executables).
 - * **clean**: removes all compiled files and executables, and restores the system to a clean state. It must also take care of cleaning up any IPC resources used by the project (e.g., named pipes).
 - * **run**: executes one (possibly out of many) scenario of the project, at your choice, using the program(s) that the project requires (e.g., the bootstrapping program, the user script, etc.). Tip: you can choose to use the `ARGS` variable to allow passing arguments to the bootstrapping program when running it (`make run ARGS "--num-cooks=5"`).
 - A Bash script, called `build.sh`, that does exactly the same things stated above for the Makefile, but using Bash instead of Make. The script must support at least the following commands:
 - * `./build.sh build`: compiles the code and produces the necessary executables to run the project.
 - * `./build.sh clean`: removes any compiled files and executables, and restores the system to a clean state.
 - * `./build.sh run`: executes a scenario of the project, at your choice, using the program(s) that the project requires.

- **Limitations.**

- You can freely work on the project with your group and ask for help from the instructors or online, but you must write the code yourself. Plagiarism will be verified using software tools and will be punished according to the university regulations.
- Any project not written in C and Bash (even C++, even if it compiles and runs correctly) will not be accepted.
- You can use third-party libraries so long as they are included in your submission and they cause no issues with compilation and execution.
- Solutions that deviate from the specifications will be penalized depending on the severity of the deviation. If you are unsure about whether a certain design choice is acceptable, please ask the instructors (only after having read the specifications again).

- **Deadline.**

- You (as a group) can choose to submit the project at any exam attempt of your choice: June, July, September (2026); January, or February (2027). Coordinating with your group members to choose the best time for you is your responsibility.
- The attempt in which you submit the project will be the attempt in which you will attend the exam, **as a group**. If you submit the project in the June attempt, you will attend the exam in June, and so on.
- In the Project submission page you can see on Moodle, you will see a deadline. That deadline is a hard deadline for the next exam attempt and is no later than five days before the exam date you see on Esse3. Late submissions will not be possible, as Moodle will automatically close the submission page at the deadline.
- Once the attempt has passed, the instructors will re-open the submission page for the next attempt, and so on.
- You can upload, delete and re-upload the project files as many times as you want. Once you are sure that your project is ready for submission, press the "Submit" button in Moodle to submit the project. That action will lock the submission page and make your submission final for that attempt.

2 Project 2026-2 - *Domotics*

2.1 Setting

This project requires the implementation of an emulated home automation system. In this system, each device **MUST** be represented by a distinct OS process.

Each device process maintains its own state and can be controlled via Inter-Process Communication (IPC) by other processes. The system must support a logical hierarchical structure of devices, where certain devices (Control Devices) can logically control other devices (Interaction Devices). However, to simplify the implementation, the OS-level process hierarchy is flat: all device processes are direct children of a single main Controller process. The logical hierarchy (who controls whom) is maintained via IPC routing rather than OS-level parent-child relationships.

Devices can be queried and commanded either centrally via the controller or locally via external manual interaction.

2.2 Specification

2.2.1 Device entity

All devices in the system, regardless of their specific type, share a base structure consisting of:

- State: a representation of the current condition of the device (e.g., on/off, open/closed).
- Switches: two-position controls (on/off) used to modify the state.
- Registry: a set of parameters and attributes specific to the device (you can add custom ones beyond the mandatory ones described below).

2.2.2 Control devices

These devices logically manage other devices linked below them. In "write" mode, a command issued to a control device is propagated via IPC to all its logically controlled children. In "read" mode, the control device shows the state of its children.

Specific information about the children **must not** be cached or saved in the controller; it must be retrieved by actively querying the child processes via IPC.

- **Controller:** Manages the entire system. It is unique, cannot be deleted, and acts as the OS-parent to all other processes.
 - State: on/off.
 - Switches: main (on/off to power the whole system).
 - Registry: **num** (number of directly connected child devices).
- **Hub:** Allows connecting multiple devices in parallel.
 - State, Switches, Registry: Mirrors the state of its children. The Hub's state is considered consistent only after a switch command has been propagated to all its children (at that point all children share the same state). If children subsequently have inconsistent states (e.g., due to a manual override), querying the Hub must report a "manual override" rather than a definite state.
- **Timer:** Allows defining a schedule to control a single connected device (or branch).
 - State, Switches: Mirrors the state of the connected device (see Hub mirroring definition above: state is consistent only after the Timer propagates a switch command to its child).
 - Registry: **begin** (activation time, format HH:MM), **end** (deactivation time, format HH:MM).

Override Resolution Rule: If child devices have conflicting states (e.g., due to local manual intervention), the parent control device must detect this inconsistency and report the presence of a "manual override". However, **any new command sent to a control device immediately overrides the current state of all its children, clearing any previous manual overrides.**

2.2.3 Interaction devices

These represent the final actuators/sensors in the system.

- **Bulb:**
 - State: on/off.
 - Switches: power (on/off).
 - Registry: **time** (total usage/on time).
- **Window:**
 - State: open/closed.
 - Switches: open and close (on/off). **Note: these instantly return to "off" after being triggered.**
 - Registry: **time** (total time left open).
- **Fridge:**
 - State: open/closed.
 - Switches: open/close (on/off).
 - Registry: **time** (time left open), **delay** (time after which it auto-closes), **perc** (fill percentage 0-100%), **temp** (current internal temperature), **thermostat** (target temperature). The content (**perc**) and the target temperature (**thermostat**) can only be modified manually.

2.2.4 Logical hierarchy and linking

Because all processes are OS-level children of the Controller, the system hierarchy is purely logical and defined by IPC routing. When the `link <id1> to <id2>` command is issued, you do NOT need to kill or restart processes. Instead, you must:

1. Send an IPC message to device `id1` informing it of its new logical parent (`id2`).
2. Send an IPC message to device `id2` (if it is a control device) informing it that it has a new logical child (`id1`).
3. The devices must update their internal routing tables/file descriptors so that future commands and state queries flow between `id2` and `id1`.

If `id2` is not a control device (i.e., not a Controller, Hub, or Timer), the link command must fail immediately with an appropriate error.

2.2.5 External interaction (manual override)

It must be possible to trigger switches and set parameters bypassing the root "Controller". This simulates a physical human action on the device (e.g., physically pressing the button on a smart bulb).

To allow a standalone executable to find and communicate with the correct target process, the system must publish routing information. For example, the root Controller could maintain a `.registry` text file mapping Device IDs to their respective PIDs/Endpoints, or devices could create strictly named IPC endpoints (e.g., a named pipe at `/tmp/domotica_bulb_3.fifo`) that the external executable can write to.

2.2.6 Concurrency

- The Controller must be able to handle commands from the interactive shell while simultaneously processing IPC messages from child devices (e.g., state updates, override notifications). The choice of concurrency mechanism (e.g., threads, `select()`, non-blocking I/O) is up to you, but you must justify your choice in the report.
- Manual overrides can happen at any time, even while the Controller is propagating a command to child devices. The system must remain consistent: a device must never be left in an undefined state.
- To allow for checking and assessing concurrency, each device must wait a random amount of time (between 1 and 3 seconds) before responding to any IPC command, to simulate processing time.

2.2.7 Error handling

- All operations in the domotics system **must** use return codes to indicate the success or failure of the operation. The system **must** define a set of error codes that can be returned by the operations, such as `DEVICE_NOT_FOUND`, `INVALID_COMMAND`, `LINK_FAILED`, `DEVICE_TYPE_MISMATCH`, etc. The error codes should be defined and used consistently by both C and Bash code, but do not need to be shared (e.g., you can `#define` the error code in C and use a variable in Bash, as long as they have the same value).
- Any parameter in any Bash and C function that can fail (e.g., due to invalid input, system errors, etc.) **must** handle the error gracefully.
- Any IPC operation (e.g., sending a command to a device, reading a response) **must** check for errors and handle them appropriately (e.g., device process crashed, pipe broken).

2.2.8 Edge cases

Check what happens when:

- A manual override is issued to a device at the exact same time the Controller sends a command to the same device.
- A device process crashes unexpectedly (e.g., killed with `SIGKILL`): the Controller must detect this and handle it gracefully.

- The user tries to link an Interaction Device as a parent of another device (only Control Devices can be parents).
- The user tries to link a device to itself, or create a cycle in the logical hierarchy (e.g., $A \rightarrow B \rightarrow A$).
- The user tries to delete a Hub that has children: the cascade deletion must terminate all logical children.
- The user issues a **switch** command to a Hub, but one of its children has crashed: the Hub must report the failure without hanging.
- The Timer's **begin** and **end** times are invalid (e.g., **begin** > **end**, or times in the past).

2.3 Interfaces

2.3.1 Controller (interactive shell)

The main process (controller) must open an interactive shell accepting the following commands:

- **list** : lists all devices, their unique ID, and summarizes their characteristics.
- **add <device>**: spawns a new device process as an OS-child of the controller (e.g., **add bulb**) and prints its details.
- **del <id>** : logically and physically terminates the device process. If it is a control device, it must send termination signals via IPC to cascade the deletion to all its logical children.
- **link <id1> to <id2>** : updates the IPC routing so that **id1** is logically controlled by **id2** (where **id2** is a Controller, Hub, or Timer).
- **switch <id> <label> <pos>** : sets the switch **label** of device **id** to position **pos** (e.g., **switch 3 power on**).
- **info <id>** : displays the complete details of the device.

2.3.2 Manual interaction

A separate executable must be provided to simulate external manual actions, run from a secondary terminal:

```
./manual_interaction.sh <id> <command> [parameters]
```

It can be freely implemented either as a Bash script or a C program, but it must be able to send commands directly to the target device process, bypassing the Controller's command routing. For example, it could write directly to a named pipe that the target device listens to.